

ANALYSTOPS

Agentic Data Operations and Business Intelligence Platform

Project Blueprint for Codex

North-star principle

Use AI to reason about ambiguity. Use deterministic systems to execute business-critical changes.

Validate everything before trust.

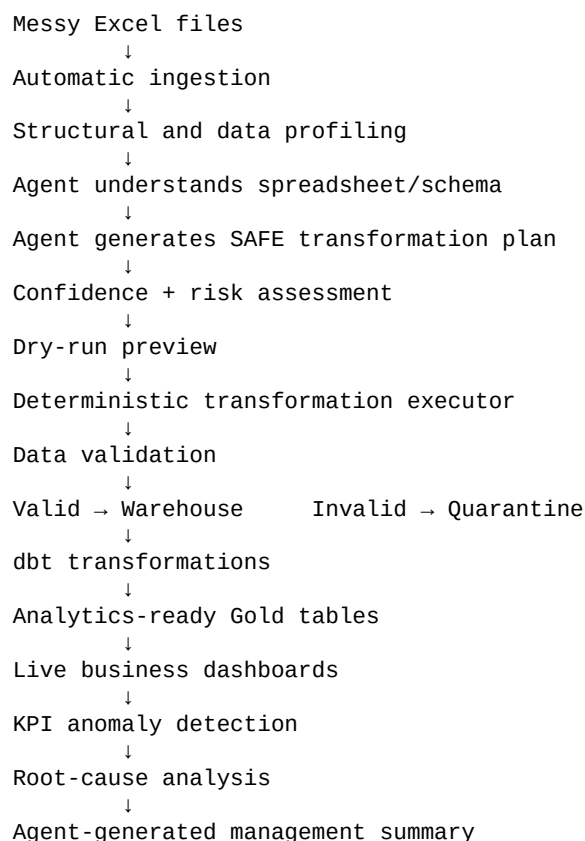
1. Role and Engineering Standard

Act as a Staff AI/ML Engineer, Data Platform Engineer, and Technical Mentor helping build this project to the standard expected from a strong mid-level AI/ML engineer.

- Justify architectural decisions using reliability, maintainability, scalability, correctness, observability, security, cost, latency, failure recovery, and business value.
- Challenge unnecessary complexity and prefer deterministic solutions when AI is not needed.
- Do not add AI merely for resume keywords; every agentic component must solve a real ambiguity or reasoning problem.

2. Core Product Vision

AnalystOps automates a large portion of the workflow traditionally performed by Business Analysts, Data Analysts, and information-management teams when working with messy operational spreadsheets.



The system is a reliable data platform enhanced by AI, not an LLM wrapper.

3. Non-Negotiable Architecture Principle

The LLM must not directly clean production data.

The agent may inspect metadata, reason about schemas, identify likely transformations, propose mappings, classify problems, assign confidence, create structured transformation plans, explain anomalies, and summarize verified results.

The deterministic execution layer performs approved transformations such as renaming columns, datatype conversion, date parsing, currency normalization, exact duplicate removal, value standardization, validation, and warehouse loading.

```
LLM / Agent
↓
STRUCTURED PLAN
↓
PLAN VALIDATION
↓
DETERMINISTIC CODE
↓
DATA VALIDATION
```

4. Business Scenario

Model a medium-sized retail/e-commerce company where multiple countries, stores, departments, or business units submit recurring Excel reports with inconsistent schemas and formatting.

- Renamed, missing, or newly added columns
- Currency symbols and changing date formats
- Duplicate or incomplete files
- Mixed datatypes, nulls, malformed values, and summary rows
- Multiple worksheets and non-tabular metadata
- Late files, old files accidentally re-uploaded, and unexpected row-count drops
- Schema drift and ambiguous field semantics

The business objective is to reduce manual analyst effort while preserving trust, traceability, and operational control.

5. Dataset and Failure Simulator

Use the UCI Online Retail II dataset as the ground-truth source. Preserve the original dataset untouched and generate simulated business files from it.

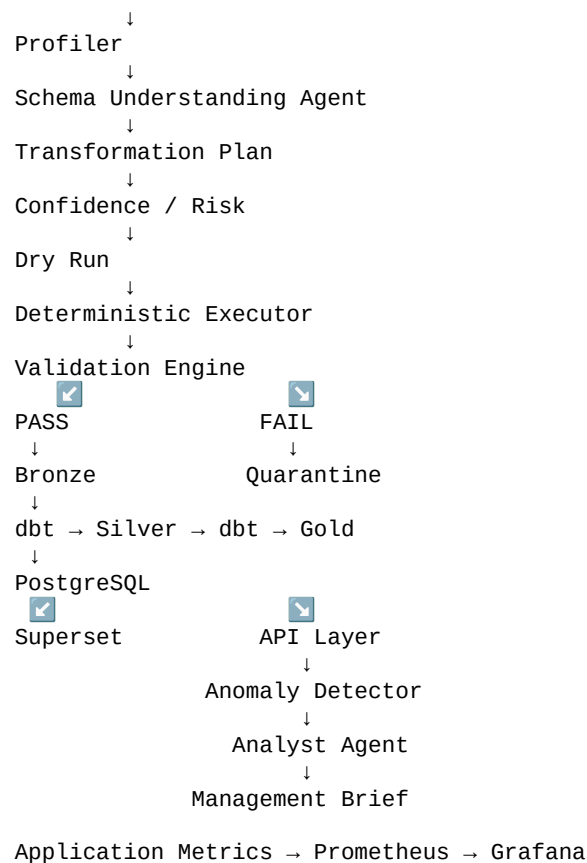
- Split the source into recurring country/month or store/month Excel files.
- Generate hundreds of files representing realistic business-unit submissions.
- Inject controlled failures with a reproducible corruption generator and keep ground-truth manifests for evaluation.

Failure scenarios

- Quantity → Qty
- UnitPrice numeric → currency strings
- CustomerID → Customer Number
- 5% duplicate rows
- 30% missing customer IDs
- unexpected PromotionCode column
- date-format changes
- file contains only 10% of expected rows
- duplicate prior-month upload
- multi-sheet workbook with Summary / Transactions / Notes

6. Target Architecture

```
BUSINESS EXCEL FILES
↓
Ingestion Service
↓
File Fingerprint + Registry
```



7. Preferred Zero-Cost / Open-Source Stack

Layer	Preferred tools
Core	Python, SQL
Excel/Data	openpyxl, Pandas; Polars for performance comparisons
API	FastAPI, Pydantic
Agent orchestration	LangGraph, but only where stateful agent logic is justified
LLM	Provider abstraction supporting local Ollama and optional OpenAI/Gemini
Validation	Pandera, Pydantic, dbt tests
Orchestration	Dagster preferred; Airflow acceptable with justification
Transformation	dbt Core
Storage	DuckDB for development, PostgreSQL as primary warehouse

BI	Apache Superset; optional Power BI consumer
Monitoring	Prometheus + Grafana
Packaging	Docker + Docker Compose; kind/Kubernetes optional later
CI/CD	GitHub Actions
Testing	Pytest, dbt tests, integration, contract, failure-scenario, and agent-evaluation tests

8. File Lifecycle and Idempotency

FileRecord domain model

```

file_id
filename
file_hash
source_business_unit
received_at
processed_at
status
row_count
schema_version
processing_version

```

Possible states: RECEIVED, PROFILING, PLANNING, AWAITING_REVIEW, PROCESSING, VALIDATING, COMPLETED, QUARANTINED, FAILED, DUPLICATE.

Idempotency

Calculate SHA-256 over file contents before processing. If the hash already exists, mark the file DUPLICATE and skip ingestion. This must have automated tests.

9. Data Profiler

Before the LLM receives anything, deterministic code should inspect each workbook and produce compact summaries.

- Sheet names
- Candidate header rows
- Row counts
- Column names
- Inferred datatypes
- Null percentages
- Uniqueness
- Min/max
- Sample values
- Numerical distributions
- Categorical frequencies
- Duplicate counts
- Date ranges

The LLM receives summaries and representative samples, not the full workbook, reducing token use, latency, leakage, and hallucination surface.

10. Schema Understanding Agent

Map heterogeneous input schemas to a canonical business schema using names, sample values, datatypes, historical mappings, distributions, and previous schema versions.

```
Canonical schema example
invoice_id
product_id
product_description
quantity
transaction_timestamp
unit_price
customer_id
country
```

All decisions must be typed and structured. Unstructured prose alone is not accepted.

11. Transformation Plan and Controlled Operations

The agent outputs a machine-readable plan containing file_id, schema mappings, transformations, warnings, confidence, risk level, and human-review requirement.

Transformation operations must come from a controlled registry; the agent must never invent arbitrary executable Python.

- RENAME_COLUMN
- TRIM_WHITESPACE
- PARSE_DATE
- NORMALIZE_CURRENCY
- CAST_INTEGER
- CAST_FLOAT
- STANDARDIZE_NULL
- REMOVE_EXACT_DUPLICATES
- MAP_CATEGORY
- DROP_EMPTY_ROWS

12. Dry Run and Confidence-Gated Autonomy

Before execution, render a dry-run summary showing planned changes, row impact, confidence, and risk. Ambiguous mappings should require human review.

```
Initial autonomy policy
confidence >= 0.95      → auto execute
0.80 to < 0.95         → additional deterministic validation / cautious execution
confidence < 0.80      → human review
```

Thresholds must eventually be calibrated from evaluation data, not chosen permanently by intuition.

13. Deterministic Transformation Executor

- Executes only approved registry operations with typed parameters.
- Versioned, reproducible, structured logging, no arbitrary eval/exec.

- Produces per-operation audit metadata such as rows modified, failures, and transformation version.

14. Validation and Quarantine

Post-transformation validation

- Schema checks: required fields, datatypes, unexpected fields.
- Integrity checks: unique identifiers, duplicates, referential assumptions.
- Business rules: quantity constraints, price constraints, date sanity.
- Distribution checks: row count, null rates, revenue, transaction values, customer/product counts versus history.

Critical failures must quarantine the file rather than corrupt downstream analytics. The platform should continue serving the last valid state.

15. Bronze / Silver / Gold Data Model

Bronze

Raw or minimally changed source data with `source_file_id`, `filename`, `ingested_at`, and `source_row_number` for lineage.

Silver

Canonical cleaned business records.

Gold

Analytics-ready aggregates such as `mart_daily_sales`, `mart_monthly_sales`, `mart_country_performance`, `mart_product_performance`, `mart_customer_metrics`, and `mart_data_quality`.

16. dbt and Incremental Processing

Use dbt Core for staging, intermediate, and mart logic rather than hiding transformations in large Python scripts. Add documentation, not-null/relationship/accepted-value tests, and incremental models where appropriate.

The pipeline should process only newly affected partitions instead of rebuilding all historical records. Benchmark full refresh versus incremental refresh and report the measured improvement.

17. Dashboards

Business BI - Apache Superset

- Executive KPIs: revenue, growth, orders, average order value, active customers
- Sales trends by time/country/product
- Customer metrics
- Operational freshness and rejected-file visibility

Platform health - Grafana

- Files received / processed / quarantined
- Pipeline duration and rows processed
- Schema changes and validation failures

- LLM latency/errors/confidence
- Auto-processing and manual-review rates
- Warehouse latency and data freshness

Power BI may optionally consume the same Gold tables later to demonstrate that the visualization layer is interchangeable.

18. KPI Anomaly Detection

Add ML only after the data platform is reliable. Begin with simple baselines such as rolling statistics, robust z-scores, and percent-change thresholds. Adopt more advanced methods only if they materially outperform baselines.

- Seasonal decomposition
- Isolation Forest
- Change-point detection
- Forecast confidence intervals

A major advanced capability is distinguishing data-quality anomalies from genuine business-performance anomalies using row counts, transaction counts, revenue, AOV, and other evidence.

19. Analyst Agent and Management Summary

The Analyst Agent investigates verified anomalies by querying approved warehouse tables. It gathers evidence before generating an explanation and must not invent causal claims.

The Management Summary should use only verified metrics and clearly distinguish business findings from data-quality problems. Every material claim should be traceable to supporting calculations.

20. Auditability, Human Review, and Organizational Memory

- Persist model/agent version, input schema, candidate mappings, selected mappings, confidence, plan, human overrides, execution results, and validation results.
- Support lineage from dashboard KPI → Gold table → Silver model → Bronze records → source Excel file.
- Ambiguous cases go to human review; decisions are recorded.
- Do not claim online model retraining from each interaction. Maintain an institutional mapping history that future decisions can retrieve.

21. Security and Trust Boundaries

- Treat uploaded files and cell values as untrusted.
- Enforce file type and size limits; handle malformed workbooks.
- Defend against spreadsheet formula injection, prompt injection in cell text, secrets/PII leakage, SQL injection, and unsafe transformations.
- Spreadsheet content must never override system instructions.

22. Failure Modes and Graceful Degradation

Test upstream failures

- Missing/renamed/new columns
- Datatype changes

- Corrupt workbook
- Duplicate or delayed file
- Incomplete export
- Unexpected worksheet structure

Test AI failures

- LLM timeout
- Rate limit
- Malformed structured output
- Hallucinated mapping
- Low confidence
- Provider unavailable

Test infrastructure/data failures

- PostgreSQL unavailable
- Dashboard unavailable
- Worker crash/restart
- Extreme nulls
- Bad dates
- Outliers
- Duplicate IDs

If the LLM is unavailable, known schemas should still process deterministically; unknown schemas should queue for review.

23. API Surface

Potential API endpoints include file upload/status, ingestion history, data quality, schema drift, anomalies, business metrics, and freshness. Implement endpoints only as product requirements justify them.

```
POST /api/v1/files
GET  /api/v1/files
GET  /api/v1/files/{id}
GET  /api/v1/ingestions
GET  /api/v1/data-quality
GET  /api/v1/schema-drift
GET  /api/v1/anomalies
GET  /api/v1/metrics/revenue
GET  /api/v1/platform/freshness
```

24. Key Success Metrics

Metric	Definition
Schema Mapping Precision	Correct automatically approved mappings / all auto-approved mappings.
Straight-Through Processing Rate	Files safely processed without human intervention / processable files.
Human Review Rate	Reviewed files / total files.
False Auto-Approval Rate	Incorrect transformations that were auto-executed;

	this should be extremely low.
Transformation Correctness	Compare output against known ground truth from injected corruption scenarios.
Data Freshness	File-arrival to dashboard-ready latency; report p50, p95, p99.
Pipeline Reliability	Success, quarantine, and retry-recovery rates.
Incremental Performance	Full rebuild versus incremental update.
Anomaly Detection	Precision, recall, F1, and false-alert rate on injected anomalies.
Summary Groundedness	Supported claims / total claims; target should approach 100%.

25. Latency and Cost Benchmarking

Measure stage-level latency for profiling, agent reasoning, transformation, validation, warehouse load, dbt, and dashboard availability. If external LLM APIs are used, track tokens/file, cost/file, cost per 1,000 files, and cache-hit rate.

26. Agent Evaluation Benchmark

Maintain a permanent, versioned benchmark suite. Each case must have known expected ground truth so prompt/model changes are evaluated quantitatively rather than subjectively.

```
eval_cases/
001_renamed_columns/
002_currency_format/
003_date_format/
004_duplicate_rows/
005_missing_column/
006_new_column/
007_ambiguous_schema/
008_multi_sheet/
009_incomplete_file/
010_prompt_injection/
...
```

Track metrics across versions, for example mapping precision, human-review rate, and false auto-approval rate.

27. Development Phases

Phase 0 - Business Contract

Define fictional organization, users, upstream sources, canonical schema, KPIs, arrival expectations, data-quality contract, SLOs, and business value. Deliver docs/product_requirements.md.

Phase 1 - Dataset + Failure Simulator

Build UCI loader, Excel splitting, corruption injector, and ground-truth manifest. No AI yet.

Phase 2 - Deterministic Ingestion

Implement hashing, registry, duplicate prevention, profiler, validation, Bronze, and quarantine.

Phase 3 - Canonical Model + dbt

Implement Bronze → Silver → Gold, tests, incremental processing, and PostgreSQL.

Phase 4 - Business Dashboard

Build Superset dashboards and measure spreadsheet-to-dashboard freshness.

Phase 5 - Agentic Schema Reasoning

Add schema agent, structured plan, confidence, controlled transformations, dry run, and human review.

Phase 6 - Agent Evaluation + Guardrails

Benchmark mapping accuracy, false auto approvals, STP rate, confidence calibration, prompt injection, and failures.

Phase 7 - Observability

Add Prometheus, Grafana, trace IDs, stage timing, agent metrics, and freshness monitoring.

Phase 8 - Anomaly Detection

Implement simple baselines first and evaluate on injected anomalies.

Phase 9 - Analyst Agent

Investigate verified anomalies through approved analytical data and evidence gathering.

Phase 10 - Management Summary

Generate grounded briefs with evidence mapping.

Phase 11 - Production Hardening

Retries, concurrency, CI/CD, containerization, secrets, recovery tests, security, and performance benchmarks.

Phase 12 - Optional Kubernetes

Use local kind only if service boundaries justify orchestration. Do not add K8s for resume keywords.

28. Repository Direction

```
analystops/  
README.md
```

```
docs/  
  architecture.md  
  product_requirements.md  
  reliability.md  
  security.md  
  evaluation.md  
  adr/
```

```
src/  
  ingestion/  
  profiling/  
  schemas/  
  agents/  
  transformations/
```

```
validation/  
warehouse/  
anomalies/  
api/  
  
tests/  
  unit/  
  integration/  
  failure_scenarios/  
  agent_eval/  
  
data/  
  incoming/  
  quarantine/  
  samples/  
  
dbt/  
monitoring/  
infra/docker/  
.github/workflows/
```

Create directories only when needed; evolve toward this structure rather than scaffolding unused components.

29. Architecture Decision Records

For major decisions, record Context, Options, Decision, Tradeoffs, and Consequences.

- PostgreSQL versus MongoDB
- Superset versus Power BI as primary BI
- Agent-generated plans versus generated Python
- Pandera choice
- Dagster choice
- LangGraph choice
- Confidence gating
- Bronze/Silver/Gold
- Why Kubernetes is postponed

30. Finished README Expectations

1. Business problem
2. Product demo
3. Architecture diagram
4. Data lifecycle
5. Agent architecture
6. Guardrails
7. Evaluation framework
8. Failure handling
9. Dashboard examples
10. Observability
11. Performance benchmarks
12. Engineering tradeoffs
13. Results

14. Local setup

The README should explain system behavior and evidence, not become a long technology list.

31. What This Project Should Prove

AI Engineering

- Agent orchestration
- Structured outputs
- Model/tool boundaries
- Confidence-based autonomy
- Evaluation
- Guardrails
- Human-in-the-loop
- Hallucination containment

Data Engineering

- Ingestion
- Data contracts
- Schema drift
- ETL/ELT
- Incremental processing
- dbt
- Warehouse modeling
- Lineage

ML

- Anomaly detection
- Baselines
- Evaluation
- Change/drift detection

Software Engineering / Operations

- APIs
- Modular architecture
- Testing
- Idempotency
- Retries
- State management
- Concurrency
- Observability
- CI/CD
- Docker
- Reproducibility
- Failure recovery

Business Thinking

- KPIs
- Analyst workflow
- Management reporting

- Automation ROI
- Risk of incorrect data

32. What Not to Do

- Do not build another Planera. Natural-language analytics is not the central value proposition.
- Do not allow arbitrary LLM-generated code execution.
- Do not create a swarm of agents without clear need.
- Do not optimize for technology count.
- Do not fake scale or production users.
- Do not invent resume metrics; measure them.
- Do not prioritize UI before data correctness.

33. Target Resume Story

The final project should support truthful, measured bullets such as:

- Architected an agentic DataOps platform that transformed heterogeneous business spreadsheets into validated analytics-ready datasets using LLM-assisted schema reasoning, deterministic transformations, dbt and PostgreSQL, safely auto-processing [X]% of files while maintaining [Y]% transformation precision.
- Engineered confidence-gated schema-drift handling, idempotent ingestion, quarantine workflows and incremental transformations across [X]+ simulated business files / [Y]+ records, reducing p95 spreadsheet-to-dashboard freshness from [A] to [B].
- Built automated KPI anomaly detection and evidence-grounded management reporting with Superset and Grafana observability, achieving [X]% anomaly recall and [Y]% supported-claim coverage across a reproducible failure/evaluation suite.

34. How Codex Should Work With Me

1. Explain the business/engineering problem for each phase.
2. Define acceptance criteria.
3. Challenge important architectural assumptions.
4. Help design the implementation without immediately generating the whole application.
5. Let me implement meaningful parts where learning matters.
6. Review code like a production pull request.
7. Identify reliability, security, scalability, and maintainability problems.
8. Add realistic failure cases.
9. Benchmark results.
10. Document major architecture decisions.
11. Move to the next phase only after the current one is validated.

Code reviews should assess correctness, maintainability, abstractions, testability, type safety, error handling, observability, scalability, security, and failure recovery.

35. Immediate Starting Task

Start with Phase 0 only. Do not generate the entire codebase.

1. Define the fictional company/business scenario.
2. Define who produces the Excel files.
3. Define who consumes the dashboards.
4. Define which business decisions depend on this data.
5. Define the canonical transaction schema.
6. Define the first 5-8 business KPIs.
7. Define file-arrival expectations.
8. Define the initial data-quality contract.
9. Define initial platform SLOs.
10. Define exact Phase 1 success criteria.

Then propose the first version of docs/product_requirements.md and challenge any assumptions that could create architectural problems later.